

an even or odd number, again using *askinteger()*. Moreover, *asksign()* checks for the sign. In case you're trying these with unknown variables, Maxima will ask the user the necessary questions to deduce the answer, and store it for future analysis. For example, *askinteger(x)* would ask you if 'x' is an integer. And, if you say 'yes', it can deduce much more by itself. Given below are some examples:

```
$ maxima -q
(%i1) askinteger(1);
(%o1) yes
(%i2) askinteger(1.0);
rat: replaced 1.0 by 1/1 = 1.0
(%o2) yes
(%i3) askinteger(1.2);
rat: replaced 1.2 by 6/5 = 1.2
(%o3) no
(%i4) askinteger(-9);
(%o4) yes
(%i5) askinteger(2/3 + 3/4 + 1/6 + 5/12);
(%o5) yes
(%i6) askinteger(-9, even);
(%o6) no
(%i7) askinteger(0, even);
(%o7) yes
(%i8) properties(x);
(%o8) []
(%i9) askinteger(x);
Is x an integer?
yes; /* This is our response */
(%o9) yes
(%i10) askinteger(x + 9);
(%o10) yes
(%i11) askinteger(2 * x, even);
(%o11) yes
(%i12) askinteger(2 * x + 1, even);
(%o12) no
(%i13) askinteger(x, even);
Is x an even number?
n; /* This is our response */
(%o13) no
(%i14) askinteger(x, odd);
(%o14) yes
(%i15) askinteger(x^3 - (x + 1)^2, even);
(%o15) no
(%i16) properties(x);
(%o16) [database info, kind(x, integer), kind(x, odd)]
(%i17) asksign((-1)^x);
(%o17) neg
(%i18) asksign((-1)^(x+1));
(%o18) pos
(%i19) asksign((-1)^x+1);
(%o19) zero
(%i20) quit();
```

Maxima simplification uses the concept of the properties of symbols. As an example, note the properties of 'x' in the above demonstration at *%i8* and *%i16*.

Complex number simplification

Complex numbers have two common useful forms, namely, the exponential and the circular (with sine and cosine) forms. *demoivre()* converts the exponential to circular form and *exponentialize()* does the reverse. Using *expand()* along with them can simplify complicated expressions. Here are a few examples:

```
$ maxima -q
(%i1) string(demoivre(exp(%i*x^2))); /* %i is sqrt(-1) */
(%o1) %i*sin(x^2)+cos(x^2)
(%i2) string(exponentialize(a*cos(t))); /* %i is sqrt(-1) */
(%o2) a*(%e^(%i*t)+%e^-(%i*t))/2
(%i3) string(expand(exponentialize(a*cos(t)+%i*sin(t)))); /* %i is
sqrt(-1) */
(%o3) a*%e^(%i*t)
(%i4) quit();
```

Expansions and reductions

As already seen in the previous article, *expand()* expands an expression completely, by default. However, it can be controlled by specifying the maximum power to which to expand for both the numerator and the denominator, respectively. Using *factor()* can compact the expanded expressions. Often, you might want to expand it with respect to only some variable(s). Say, $(x + a + b)^2$ should be expanded with respect to 'x'. *expandwrt()* is meant exactly for that purpose. One example for each of these scenarios is shown below:

```
$ maxima -q
(%i1) string(expand(((x+1)^2-x^2)/(x+1)^2, 2, 0));
(%o1) 2*x/(x+1)^2+1/(x+1)^2
(%i2) string(factor(((x+1)^2-x^2)/(x+1)^2));
(%o2) (2*x+1)/(x+1)^2
(%i3) string(expandwrt((x+a+b)^2,x));
(%o3) x^2+2*(b+a)*x+(b+a)^2
(%i4) quit();
```

Expressions containing logs, exponentials and radicals (powers) can be simplified using *radcan()*. Rule-based simplifications can be achieved using the sequential comparative simplification function *scsim()*. Both of these call for a few examples:

```
$ maxima -q
(%i1) string(radcan(exp(5 * log(x) + log(3 * exp(log(y) / 4)))));
(%o1) 3^x/5*y^(1/4)
(%i2) radcan((log(2*x+2*x^2)-log(x))/(log(1+1/x)+log(2*x)));
(%o2) 1
(%i3) expr: a^2 + b^2 + c^2$
(%i4) eq1: a^2 + 2*a*b + b^2 = 4$
(%i5) eq2: a * b = 6$
```